

INGEGNERIA DEL SOFTWARE

- APPUNTI LEZIONI

02.PROCESS

Il processo software è l'insieme delle attività, tecniche e strumenti utilizzati per produrre, verificare e mantenere il software.

Si compone di tre categorie fondamentali di attività:

- produzione (requirements, design, implementation)
- validazione e verifica (V&V)
- gestione (project management, configuration management, quality assurance).

L'obiettivo è ottenere **software funzionante**, affidabile e sviluppato secondo **tempi e costi** pianificati. La produzione parte dal codice eseguibile, ma in realtà si scrive codice sorgente, che richiede organizzazione logica (funzioni, classi, package) e progettazione. Perciò il lavoro parte dai requisiti, poi si passa alla progettazione dell'architettura e infine all'implementazione. **Ogni fase dipende dalla precedente.**

Il processo prevede **documenti specifici**: il documento dei requisiti, il documento di design, il codice stesso. A queste fasi si affiancano le attività di verifica e validazione, che controllano la coerenza interna ed esterna di requisiti, design e codice, e confermano che il software risponda alle esigenze dell'utente.

La gestione comprende:

- Project Management: pianificazione, assegnazione delle attività, controllo costi e tempi.
- Configuration Management: identificazione, tracciamento e conservazione dei documenti e componenti.
- Quality Assurance: definizione e controllo degli standard qualitativi.

Il processo software non si esaurisce con lo sviluppo. Le fasi successive comprendono operazione, **manutenzione** e infine il ritiro del software. La manutenzione è spesso più costosa e lunga dello sviluppo iniziale, perché ogni modifica è vincolata dalle scelte passate. Esistono diversi scenari di utilizzo:

- Sistemi informatici aziendali: lunghi tempi di sviluppo e manutenzione prolungata.
- Software consumer (es. videogiochi): sviluppo breve, minima manutenzione.
- Sistemi operativi: sviluppo e manutenzione lunghi e complessi.

In sintesi, il processo software fornisce un quadro di riferimento fondamentale per produrre software di qualità, organizzando le attività in modo razionale e ripetibile.

03.REQUIREMENTS

L'ingegneria dei requisiti è la fase del processo software in cui si definiscono le caratteristiche del prodotto prima di iniziarne lo sviluppo. Una corretta definizione dei requisiti è fondamentale: se mancante o imprecisa, può compromettere la qualità del prodotto, rendere difficile la verifica e portare a risultati insoddisfacenti.

I requisiti possono essere di tipo:

- **Funzionale**: cosa deve fare il software.
- **Non funzionale**: come lo deve fare (es. prestazioni, sicurezza, usabilità, portabilità).
- **Di dominio**: derivano dallo specifico ambito applicativo e ne riflettono le regole.

Il processo di ingegneria dei requisiti coinvolge vari attori (stakeholder), tra cui utenti, committenti, amministratori, esperti di dominio e sviluppatori. Questi apportano differenti punti di vista, che vanno integrati nel documento dei requisiti.

Tecniche utilizzate:

- Elicitation: interviste, focus group, osservazioni (etnografia), questionari.
- Formalizzazione: uso di tabelle dei diritti, contesti, diagrammi, identificazione chiara e numerazione dei requisiti.
- Verifica e Validazione: ispezioni, prototipi, MVP (Minimum Viable Product), iterazioni.

Strumenti utili possono essere software specializzati (es. RequisitePro), strumenti di modellazione (UML), o anche semplici fogli Word/Excel.

Elementi principali del documento dei requisiti:

1. Descrizione generale del sistema.
2. Stakeholder coinvolti.
3. Diagramma di contesto e interfacce.
4. Requisiti, suddivisi in:
 - Funzionali (identificati con codice F1, F2, ecc.)
 - Non funzionali (NF1, NF2, ...)
 - Di dominio
5. Diagramma dei casi d'uso (UCD)
6. Scenari: sequenze temporali di interazione con il sistema.
7. Glossario dei termini.
8. Design del sistema: componenti hardware/software coinvolti.

I casi d'uso (Use Cases) rappresentano gruppi di scenari che rispondono a un obiettivo dell'utente. Sono collegati agli attori (umani o sistemi esterni) e si arricchiscono con relazioni come "include" o "extend" per modularità e variabilità.

I requisiti devono essere **misurabili e testabili**. Frasi vaghe come "il sistema deve essere facile da usare" sono inutili se non accompagnate da criteri oggettivi.

La priorità tra questi requisiti è spesso una decisione aziendale, non tecnica.

04.UML

UML (Unified Modeling Language) è un linguaggio standardizzato per la modellazione di sistemi software e concettuali. Creato per supportare la progettazione orientata agli oggetti, è stato definito dall'OMG (Object Management Group) e comprende diversi tipi di diagrammi, ognuno con uno scopo specifico.

CLASS DIAGRAM

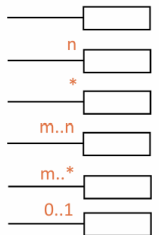
Il diagramma delle classi è utile per rappresentare concetti del dominio, strutture software, sistemi hardware e componenti.

Concetti principali:

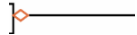
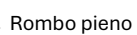
- Classe: rappresenta un insieme di oggetti con attributi e comportamenti comuni.
- Oggetto: istanza concreta di una classe.
- Attributo: proprietà elementare associata a una classe.
- Operazione: comportamento eseguibile dagli oggetti.
- Associazione: relazione tra due classi (es. uno studente è iscritto a un corso).

Le **associazioni** possono essere:

- Semplici: legano due classi (es. Studente-Corso).
- Ricorsive: coinvolgono la stessa classe (es. Persona-è-genitore-di-Persona).
- Con ruolo: specificano la funzione della classe nell'associazione.
- Con molteplicità: indicano quanti oggetti partecipano a una relazione (es. 0..*, 1..1).



Esistono tre **specializzazioni** delle associazioni:

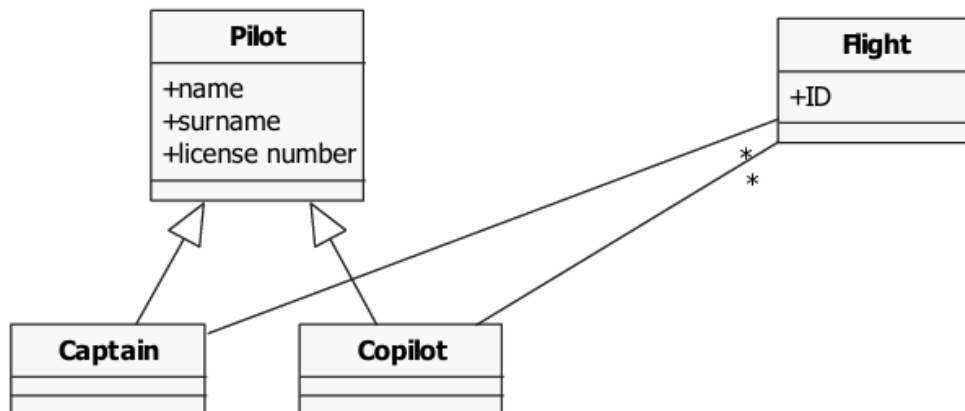
- Aggregazione: relazione "parte di", ma con vita indipendente. 
- Composizione: relazione più forte, la parte non può esistere senza il tutto.  Rombo pieno
- Classe di associazione: permette di aggiungere attributi a una relazione. 

Un altro concetto chiave è **l'ereditarietà** (specializzazione/generalizzazione):

- Una classe derivata *subclass* eredita attributi e associazioni della classe base *superclass*.
- È fondamentale che la relazione "è un/a" (is-a) sia valida.

Buone pratiche:

- Usare nomi singolari per le classi.
- Nominare le associazioni con verbi.
- Rappresentare le relazioni con molteplicità corretta.
- Non rappresentare eventi come associazioni (es. "entra", "esce").
- Evitare di ripetere come attributo ciò che è già espresso da un'associazione.

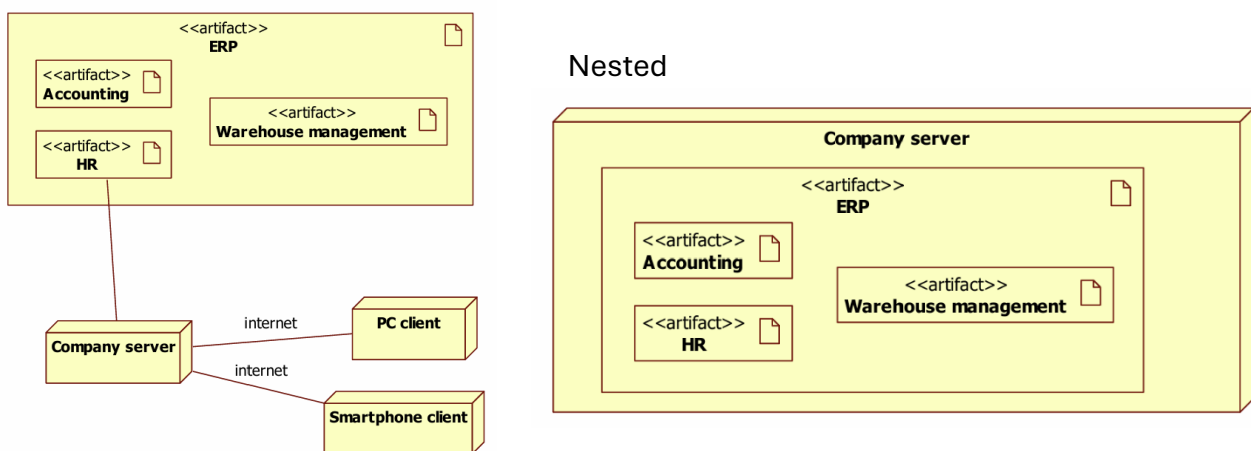


DEPLOYMENT DIAGRAM

Il diagramma di deployment serve invece a rappresentare la distribuzione fisica delle componenti software (artifacts) su entità hardware (nodes).

I concetti più importanti sono:

- Un node è un'entità fisica o virtuale in grado di eseguire software.
- Un artifact rappresenta un file, un eseguibile, una libreria, o una tabella.
- I diagrammi di deployment possono essere annidati per rappresentare la gerarchia.



UML consente quindi di modellare sia il dominio concettuale che l'architettura tecnica di un sistema. Ogni diagramma ha un suo ruolo e contribuisce alla chiarezza, alla coerenza e alla manutenibilità del progetto software.

Nella progettazione è importante scegliere cosa rappresentare: un modello concettuale, un modello di sistema o un modello software. A seconda del livello, si usano classi diverse e con gradi di dettaglio diversi.

SYSTEM DESIGN

Tutto ciò che è nel riquadro BB del context diagram fa parte del system diagram.

La parte software del system diagram poi diventa artefatto del deployment mentre la parte hardware diventano i nodi del deployment.

05.CONFIGURATION MANAGEMENT

La Configuration Management (CM) è un'attività fondamentale nello sviluppo software, necessaria per gestire il lavoro simultaneo di più persone su documenti e codice, assicurare tracciabilità delle modifiche e mantenere coerenza tra i vari artefatti del progetto.

Ogni elemento sotto controllo è detto Configuration Item (CI), che può essere un documento, un file di codice, un test, ecc. L'insieme coerente di più CIs è una Configuration.

Tutti i CIs vengono salvati in un repository, che può essere locale, centralizzato o distribuito.

La CM permette:

- Versionamento: recuperare versioni passate dei file.
- Controllo delle modifiche: stabilire chi può leggere o modificare cosa.
- Tracciamento: sapere chi ha fatto cosa e quando.
- Gestione delle dipendenze: tra requisiti, design, codice e test.

Modelli di **controllo** delle modifiche:

- Lock-modify-unlock: un solo utente alla volta modifica un CI.
- Copy-modify-merge: utenti lavorano in parallelo, poi si uniscono le modifiche (merge)

Un sistema CM (CMS) deve offrire **funzionalità** come:

- Ripristino di versioni precedenti.
- Confronto tra versioni.
- Controllo degli accessi.
- Storico delle modifiche.

Sistemi di Versionamento:

- Locali: solo su una macchina (es. RCS, ormai obsoleti).
- Centralizzati: un server e più client (es. SVN).
- Distribuiti: ogni utente ha una copia del repository (es. Git).

GIT

Git è un **CMS distribuito**, progettato per velocità, gestione di molti branch e integrità dei dati. Usa il modello a snapshot, dove ogni commit rappresenta lo stato completo del progetto in un dato momento.

Comandi fondamentali:

- git init, git clone: inizializzare o copiare un repo.
- git add, git rm: aggiungere o rimuovere file dal tracking.
- git commit: salvare una versione nel repo locale.
- git push, git pull: sincronizzare con il repo remoto.
- git status, git log, git diff: ispezionare lo stato e la cronologia.

Stato dei file (CI):

- Untracked: non monitorato.
- Staged: aggiunto per il prossimo commit.
- Committed: salvato nel repository.

Branch e Merge:

- I branch consentono sviluppo parallelo di funzionalità.
- Il merge unisce branch diversi, creando una merge commit se necessario.
- I conflitti vanno risolti manualmente.

Rebase:

- Alternativa al merge.
- Riscrive la storia del branch applicando le modifiche sopra un altro branch.
- Può semplificare la cronologia, ma va usato con cautela.

Collaborazione:

- Shared repository: accesso diretto con permessi.
- Fork and pull: ogni sviluppatore lavora su una copia, propone modifiche con una pull/merge request.

Convenzioni utili:

- Conventional commits: standard per rendere i messaggi di commit più chiari (es. feat:, fix:, BREAKING CHANGE:).

Best practice:

- Non modificare direttamente la main branch.
- Usare merge request per ogni consegna.
- Coordinare i branch nel team (es. per persona, per task).

La gestione della configurazione con Git consente controllo, tracciabilità, collaborazione efficace e facilita la manutenzione a lungo termine dei progetti software.

05_bis.GITFLOW

Il modello GitFlow nasce come un approccio strutturato alla gestione del codice sorgente nei progetti software, particolarmente utile nei contesti Agile. Non si tratta di un insieme rigido di regole, ma di una guida che molte aziende adattano secondo le proprie esigenze. L'obiettivo principale di GitFlow è quello di separare il codice stabile da quello in sviluppo, permettendo al tempo stesso il lavoro parallelo di più sviluppatori su funzionalità diverse, migliorando la qualità del prodotto attraverso test e revisioni del codice.

GitFlow prevede l'utilizzo di **branch persistenti** e **branch effimeri**. I branch persistenti sono:

- **main**: contiene solo codice stabile e pronto per la produzione. Ogni rilascio viene marcato con un tag (es. v1.0.0).
- **dev**: rappresenta l'ambiente principale di sviluppo. Le nuove funzionalità vengono integrate qui prima di essere testate.
- **QA**: utilizzato dal team di Quality Assurance per l'esecuzione dei test di integrazione e regressione.

L'uso dei branch effimeri consente di mantenere i branch principali puliti e ordinati. Ogni intervento di sviluppo, correzione o modifica viene gestito su un branch dedicato, che verrà poi eliminato dopo il merge:

- **feature/**: utilizzato per sviluppare nuove funzionalità. Parte da dev e, una volta approvato, vi viene reintegrato.
- **hotfix/**: serve a correggere bug critici in produzione. Parte da main e, una volta risolto il problema, viene reintegrato in main, e se necessario anche in dev e QA.
- **bugfix/**: impiegato per correggere bug rilevati durante i test. Parte da QA e, una volta risolto, vi viene reintegrato, eventualmente anche in dev.

Fondamentale è l'utilizzo delle **Merge Request**, che favoriscono la revisione del codice, l'approvazione collaborativa e il controllo della qualità.

Tra le buone pratiche da seguire vi sono:

- Mantenere il branch main stabile e protetto.
- Scrivere test prima di effettuare merge su dev.
- Seguire una convenzione di nomenclatura coerente (es. feature/1234-nome-funzionalità).
- Automatizzare build e test tramite pipeline CI/CD.

Tra gli errori più comuni da evitare si segnalano:

- Lavorare direttamente su branch principali senza passare per un branch effimero.
- Non aggiornare dev con le modifiche presenti in main.
- Dimenticare di cancellare i branch dopo il merge.
- Utilizzare nomi generici o personali per i branch.

In sintesi, GitFlow è uno strumento utile per gestire lo sviluppo software in modo ordinato, garantendo qualità, tracciabilità e facilitando il lavoro di squadra, soprattutto in progetti strutturati e con rilasci periodici.

06.UI

Il design dell'interfaccia utente (UI Design) è una fase cruciale nello sviluppo software, soprattutto quando sono coinvolti attori umani. La UI è responsabile dell'interazione tra utente e sistema, e va progettata sulla base dei requisiti funzionali e del contesto d'uso.

Messaggi chiave:

- La UI va testata con utenti reali.
- Le scelte vanno basate su esperimenti, feedback e scenari concreti, non su opinioni personali.

Contesto e variabili:

- Un'applicazione può avere molte UI diverse (web, mobile, desktop, smartwatch).
- Le interazioni variano: schermo, tastiera, voce, touch, eye-tracking, gesti, ecc.
- La complessità della UI dipende dalla complessità del sistema e delle sue funzioni.

Principi guida:

- Semplicità: la UI deve essere semplice quanto possibile, compatibilmente con le funzionalità.
- Adattamento agli utenti: tramite personas (profili tipo degli utenti) e storie (scenari d'uso), si progettano funzionalità e interazioni su misura.
- Usabilità e User Experience (UX): comfort, emozioni, valori.
- Design centrato sull'utente (UCD):

Activity#	Activity	Techniques
1	Identify the users	Context diagram, personas / actors
2	Define requirements	Use cases, scenarios, functional requirements
3	Define system and interactions	Prototypes
4	In lab tests	Ethnographics, Interviews
5	In field tests	A/B testing Measurements

Prototipazione:

- Low fidelity: schizzi, storyboard, carta e penna.
- High fidelity: mockup eseguibili (es. Figma, Balsamiq).
- Feedback raccolto tramite esperti, utenti reali e strumenti di analisi.

Linee guida per l'usabilità:

- Stile e formattazione coerenti tra le pagine.

- Non chiedere più volte le stesse informazioni.
- Fornire feedback chiaro (clic, caricamenti, errori, successi).
- Oggetti interattivi evidenti.
- Messaggi chiari e navigazione visibile (breadcrumb, menù).
- Layout semplice: pochi colori, pochi font, elementi ben distribuiti.

Portabilità:

- Progettare la GUI tenendo conto dei diversi ambienti (Windows, iOS, Android, Web).
- Utilizzare tecnologie cross-platform quando possibile (Flutter, Cordova, Xamarin).

ESEMPI_Vedere slides

- Analisi di casi d'uso (signup, creazione stazione di servizio, segnalazione prezzo).
- Confronto tra prototipi buoni e cattivi: errori comuni, elementi mancanti, coerenza con lo scenario.

La UI è il punto di contatto tra utente e tecnologia. Per questo motivo va progettata con attenzione, testata in modo iterativo e adattata in base ai risultati reali. L'obiettivo non è solo la funzionalità, ma anche l'efficacia comunicativa e l'esperienza dell'utente.

ARCHITECTURE AND DESIGN

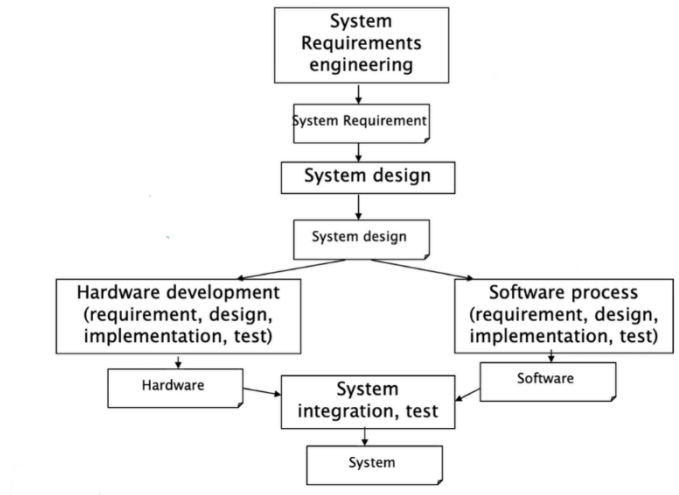
Scrivere codice senza struttura = se funziona sono solo molto fortunata

ATTENZIONE: Architectural pattern != Design Pattern

Architectural pattern = livello macro

Design pattern = arrivo al codice

UML Deployment diagram mi dice dove
va posizionato il software in mezzo
all'hardware.



DESIGN

Analisi: Architettura, design ad alto livello, design a basso livello

- Input: Requirement Document
- Output: Design Document → Componenti e connessioni x soddisfare i requisiti

Architettura: Definire ad alto livello i componenti e le loro interazioni.

Proprietà: Funzionali (Efficienza, usabilità, sicurezza, ecc.), non funzionali (Testabilità, Trasferimento, Complessità, Coupling, Costi, Schedule e Staff, ecc.)

NOTA: Coupling vuol dire che c'è connessione anche nel codice, se cambio una cosa questa va cambiata da molte altre parti (se c'è coupling)

RICORDA: Trade offs

UML: Package diagram → Packages e interazioni

Quando parliamo di **architettura** inseriamo solo il nome del package ma quando parliamo del **design** andremo a definire anche i metodi e gli attributi che saranno poi codice vero e proprio. Quello che abbiamo fatto fino ad ora è statico, ciò che è dinamico viene definito dai **state chart** (evoluzione nel tempo) e **sequence diagram** (ordine e modo in cui uso le funzioni).

ARCHITETTURA, altissimo livello

Patterns architetturali: Soluzioni già testate in passato che risolvono problemi ricorrenti.

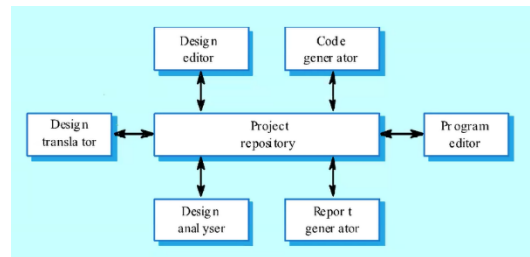
REPOSITORY

Ipotizziamo che il nostro sistema ha due sottosistemi che comunicano tra loro; quindi, abbiamo bisogno o di un db in comune o di una comunicazione tra i due db separati.

Utilizzare un repository vuol dire che i vari sotto-componenti parlano NON tra loro ma con un unico repository che ha un suo db. Quindi tra loro i componenti sono sconosciuti e non sanno da chi provengono i dati.

V: unico standard per tutti, scalabile e centralizzato. Favorisce testabilità e manutenibilità.

S: unico point of failures, può introdurre duplicazione di logica tra il repository e i modelli.

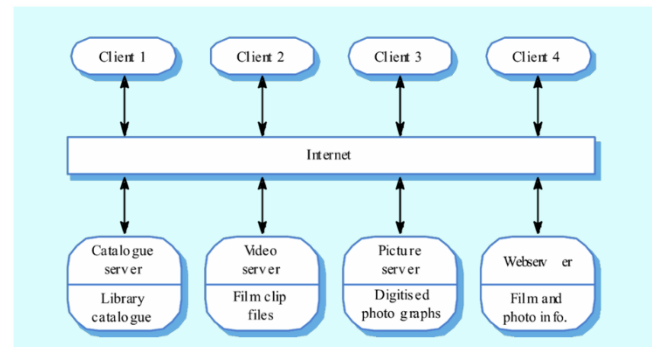


CLIENT-SERVER

In questa architettura, i client richiedono servizi a uno o più server, che li elaborano e restituiscono un risultato. Il server è generalmente centralizzato e contiene logica, dati e servizi; il client si limita all'interfaccia utente e all'invio di richieste. È la base di molte architetture web e mobile.

V: Centralizzazione della logica e dei dati, gestione più semplice della sicurezza. Consente scalabilità orizzontale con più client.

S: Dipendenza dalla connessione di rete e dalla disponibilità del server, i client sono fortemente dipendenti dai servizi esposti.



LAYERED

L'architettura a livelli divide il sistema in più strati logicamente distinti, ciascuno con una responsabilità specifica. Il livello superiore invoca i servizi del livello sottostante. Ogni livello interagisce solo con quello immediatamente inferiore o superiore, creando una struttura.

V: separazione delle responsabilità, sistema più comprensibile, facile sostituire solo un livello

S: introdurre overhead nelle chiamate tra strati.

PIPES AND FILTERS

Il sistema è composto da una catena di filtri indipendenti, ognuno dei quali esegue una trasformazione sui dati in ingresso e produce un output. I dati fluiscono da un filtro all'altro attraverso pipe (condotti). È molto usato nel trattamento di flussi dati, ad esempio in compiler o elaborazioni audio/video.

V: Filtri facilmente riutilizzabili e componibili, esecuzione parallela o distribuita

S: Gestione dello stato interno complesso è difficile, richiede che i dati siano serializzabili

BROKER

Il Broker è un intermediario che coordina la comunicazione tra componenti distribuiti in modo trasparente. I client non conoscono la posizione o la logica dei server: si connettono al broker, che instrada le richieste al componente appropriato. È spesso usato in middleware, sistemi distribuiti e architetture service-oriented (SOA).

V: facilita l'interoperabilità, supporta componenti eterogenei e distribuiti.

S: Broker single point of failure/collo di bottiglia, gestione complessa

MVC (Model-View-Controller)

Il pattern MVC separa un'applicazione in tre componenti:

- Model: gestisce i dati e la logica di business.
- View: mostra l'interfaccia utente.
- Controller: gestisce gli input dell'utente, li interpreta e aggiorna Model e View.

Questa separazione facilita la manutenibilità e lo sviluppo parallelo frontend/backend.

V: modularità, permette un'evoluzione indipendente dei componenti

S: disciplina progettuale complessa da implementare per piccoli progetti

MICROKERNEL (Plug-in architecture)

In questa architettura, un nucleo centrale (microkernel) fornisce le funzionalità base del sistema. Altre funzionalità vengono aggiunte come plugin che si integrano dinamicamente con il kernel. È utile in software configurabile o estendibile (es. IDE come Eclipse).

V: modularità e estendibilità, Plugin sviluppabili e deployabili separatamente

S: design molto robusto e flessibile, gestione complessa della compatibilità tra plugin

MICROSERVICES

L'architettura a microservizi suddivide un'applicazione in piccoli servizi indipendenti, ciascuno con una responsabilità specifica e un database autonomo. Comunicano via API REST, gRPC, o code di messaggi. Ogni microservizio può essere sviluppato, scalato e distribuito in modo indipendente.

V: Scalabilità orizzontale, facilità nel deploy continuo, resilienza ai guasti

S: Complessa orchestrazione e gestione, rischio di duplicazione della logica

DESIGN

Possiamo separare il design ad alto livello e quello a basso livello.

Alto livello: definiamo il nome delle classi con il glossario e diciamo le interazioni col sistema dei vari attori tramite il context diagram. Definiamo uno scheletro del codice ancora astratto.

Basso livello: Codice vero e proprio con classi e metodi implementati. Qui definiamo i tipi degli attributi, la loro privacy, i metodi delle singole classi, la persistenza.

Design pattern di 3 tipi:

- **Creational** :Si occupano della creazione di oggetti, cercando di rendere il codice più flessibile nel decidere quali oggetti creare, quando e come.

Esempi: Singleton, Factory Method, Abstract, Factory, Builder, Prototype

- **Structural** : Si concentrano sulla composizione delle classi o degli oggetti, facilitando la creazione di strutture più grandi e flessibili.

Esempi: Adapter, Decorator, Composite, Proxy, Facade, Bridge, Flyweight

- **Behavioral** : Si focalizzano su come gli oggetti interagiscono tra loro e su come si distribuisce il comportamento tra gli oggetti.

Esempi: Observer, Strategy, Command, State, Chain of Responsibility, Mediator, Iterator,

Scope→ indica dove agisce il pattern:

- Class: riguarda le relazioni tra classi, spesso stabilite tramite ereditarietà. Le relazioni sono statiche (note a compile-time).
- Object: riguarda le relazioni tra oggetti, che sono più flessibili e dinamiche (note a runtime).

CREATIONAL

Factory Method	Class	Definisce un'interfaccia per creare oggetti, ma lascia alle sottoclassi l'istanziamento.
Abstract Factory	Object	Fornisce un'interfaccia per creare famiglie di oggetti correlati senza specificare le classi concrete.
Prototype	Object	Crea nuovi oggetti clonandone altri esistenti, senza passare per il costruttore.
Singleton	Object	Garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale.

STRUCTURAL

Adapter	Class/Object	Converte l'interfaccia di una classe in un'altra attesa dal client.
Bridge	Object	Separa un'astrazione dalla sua implementazione permettendo di variarli indipendentemente.
Composite	Object	Compone oggetti in strutture ad albero per rappresentare gerarchie parte-intero.
Decorator	Object	Aggiunge responsabilità a un oggetto in modo dinamico, senza modificare la classe originale.
Facade	Object	Fornisce un'interfaccia semplificata a un insieme complesso di classi o sottosistemi.
Flyweight	Object	Riduce l'uso della memoria condividendo il più possibile dati tra oggetti simili.
Proxy	Object	Fornisce un surrogato o placeholder per controllare l'accesso a un altro oggetto.

BEHAVIOUR

Chain of Responsibility	Object	Permette a più oggetti di gestire una richiesta senza specificare il ricevente.
Command	Object	Incapsula una richiesta come oggetto, permettendo di parametrizzare azioni e code.
Interpreter	Class	Definisce una grammatica e un interprete per interpretare frasi nel linguaggio.
Iterator	Object	Fornisce un modo per accedere sequenzialmente agli elementi di una collezione.
Mediator	Object	Definisce un oggetto che incapsula le interazioni tra oggetti, promuovendo il disaccoppiamento.
Memento	Object	Salva e ripristina lo stato interno di un oggetto senza violarne l'incapsulamento.
Observer	Object	Definisce una dipendenza uno-a-molti, notificando i cambiamenti di stato a tutti gli osservatori.
State	Object	Permette a un oggetto di cambiare comportamento quando cambia il suo stato interno.
Strategy	Object	Definisce una famiglia di algoritmi intercambiabili incapsulati in oggetti separati.
Template Method	Class	Definisce la struttura di un algoritmo, delegando alle sottoclassi alcuni passaggi.
Visitor	Object	Consente di aggiungere nuove operazioni a oggetti senza modificarne le classi.

VALIDATION & VERIFICATION

Definizione di V&V

- **Validation**: *Stiamo costruendo il prodotto giusto?* – Verifica se il software soddisfa le reali esigenze degli utenti.
- **Verification**: *Stiamo costruendo il prodotto correttamente?* – Controlla che il software sia stato costruito secondo le specifiche tecniche.

Fasi principali del ciclo di vita software

1. Requirements definition
2. Design
3. Implementation
4. Deployment
5. Operation
6. Maintenance
7. Retirement

Errori e costi

- Defect: può essere un *fault* (errore nel codice o nei requisiti) o un *failure* (comportamento anomalo in esecuzione).
- Più tardi si scopre un difetto, più costoso è correggerlo.
- Il rework evitabile rappresenta fino al 40-50% del costo di sviluppo.

Cost of Quality

- Cost of quality = ispezioni, testing.
- Cost of non-quality = fixing difetti in produzione.
- Chi paga? Spesso l'utente, soprattutto in caso di "lock-in" del fornitore.

Technical Debt: Scelte di bassa qualità che riducono i tempi oggi, ma costano di più domani (es. codice duplicato, nomi poco chiari).

Tecniche di V&V

Statiche: Ispezioni (su requisiti, design, codice), Analisi del codice

Dinamiche: Testing (unitario, integrazione, sistema)

Ispezioni:

- Esaminano documenti o codice senza esecuzione.
- Usano la dinamica di gruppo (moderatore, scribe, lettori, autore).
- Metodo Fagan: pianificazione → lettura → incontro → rework → follow-up.
- Tecniche: checklist, reading basato su scenari, defect taxonomies.

Testing

- Processo dinamico per rilevare *failure* confrontando output atteso e reale (oracle).
- Tipi:
 - Unit test (funzioni, classi)
 - Integration test (più moduli)
 - System test (intero sistema)
 - Regression test (dopo modifiche)

NOTA: i testing non può dimostrare assenza di errori, solo scoprirli (Dijkstra).

➔ Testing efficace = uso di criteri validi e affidabili per selezione dei test.

UNIT TEST

Black Box vs White Box

- Black box: basato su requisiti (equivalence classes, boundary analysis).
- White box: basato sulla struttura del codice (statement, decision, path, loop coverage).

All'esame ci sarà un pezzo di codice e delle caratteristiche e si chiede quanti test vanno fatti per avere la percentuale di coverage più alta. Quindi max coverage possibile: ... con i test

BLACK BOX

Random:

Pro: Indipendente dai requisiti

Contro: Richiede molti casi di test (facile definire gli input, richiede a Oracle di calcolare l'output previsto)

Equivalence Class Partitioning

- Divide lo spazio degli input in classi di comportamento simile.
- Almeno un test per ogni classe, soprattutto quelle non valide.

Una classe corrisponde a un insieme di input validi o non validi per un predicato sulle variabili di input. Se un test in una classe non ha successo, gli altri test nella stessa classe possono avere lo stesso comportamento.

- Valido: accettabile
- Non valido: non accettabile (produce eccezione, errore)

Criteria, predicates, partitions:

- Criterio: attributo
- Predicato: condizione sull'attributo, definisce una partizione (== classe eq)

Ogni classe di equivalenza deve essere coperta da almeno un caso di test.

Un caso di test per ogni classe di input non valida. Ogni caso di test per classi di input valide deve coprire il maggior numero possibile di classi valide (rimanenti).

Quando un modulo ha uno stato, questo deve essere considerato per definire le **partizioni**.

Lo stato può essere difficile da leggere/creare, richiede una sequenza di chiamate.

Test of OO class ➔ ogni volta che abbiamo lo state.

WHITE BOX

L'obiettivo è garantire che tutte le parti del programma vengano eseguite almeno una volta durante i test, valutando così la copertura del codice.

→ **Statement coverage**: La statement coverage verifica se ogni istruzione del programma viene eseguita almeno una volta.

Si effettuano i test solo sugli statement del codice.

Per raggiungere il 100% di copertura, è necessario identificare quali valori di input permettano di eseguire tutte le istruzioni.

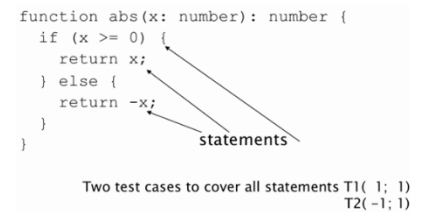
Nell'esempio qui fornito, basta eseguire due test, uno per il ramo vero e uno per quello falso (quindi 1 e -1).

Bisogna quindi capire quanti test servono per avere la più alta coverage possibile.

Per la misura dello statement coverage: $\text{Numero statement eseguiti} / \text{numero statement totali}$. È inoltre utile andare a creare un **control flow graph** della mia funzione, in questo modo riesco a vedere il comportamento del mio codice.

Devo ragionare su:

- Nodo $\Rightarrow$ Rappresenta statement o decisione.
- Archi $\Rightarrow$ Flussi di controllo tra i nodi.



COVERAGE

- Percentuale di entità coperte dai test (statement, decisioni, requisiti, funzioni).
- Obiettivo: massimizzare copertura con pochi test ben scelti.

→ Non è sempre possibile testare un intero codice, o non ha senso se per esempio uso una libreria esterna. Non vado a testare la parte della libreria che non uso.

Node coverage = $\text{number of nodes executed} / \text{total number of nodes}$

*nei casi molto semplici capita che il node coverage = state coverage.

NOTA: Se capita all'esame fallo notare.

Se calcolo tutti i rami, implicitamente copro tutti i nodi. Viceversa non è detto.

Quando abbiamo condizioni multiple (espressioni booleane complesse) può essere più faticoso farlo a mano. Come lo gestiamo?

- **Simple condition coverage**: verifica che la test suite abbia verificato ogni condizione almeno una volta. Ovviamente faccio questa se la condizione è **UNICA**.
- **Multiple condition coverage**: verifico che tutte le combinazioni siano coperte

Cosa scelgo dipende dal test che voglio fare e cosa sto testando.

Se faccio il 100% multiple condition coverage ho fatto anche simple, decision e Statement. MA se ho fatto simple condition coverage non è detto che ho fatto decision e Statement.

PATH coverage: devo contare tutti i percorsi/cicli. **Path** $\rightarrow$ sequenza di nodi nel grafico

NOTA: con un grafo ciclico devo fare dei test che abbiano comportamenti diversi sui loops.

Come faccio coverage in caso di loop? Seleziono i test in modo da testare le condizioni sui loop. Creiamo un limite sulle iterazioni e proviamo una o più iterazioni.

LoopCoverage:

- Non entra nel loop
- Entra e cicla una sola volta
- Entra e cicla più volte

NOI? Usiamo diversi tools come **Jest**.

INTEGRATION TEST

- Bottom-Up: test dei moduli di basso livello → aggregazione.
- Top-Down: test dei moduli di alto livello con stubs.
- Big Bang: tutto insieme (da evitare).

Testiamo sostanzialmente le dipendenze che ci sono tra le classi e testiamo le funzioni dipendenti. Quelle che non dipendono da altre funzioni saranno state già testate negli unit. Avremo quindi un grafico delle dipendenze.

Come testo una funzione che è in mezzo al grafico come function2 e che quindi dipende da function1 e ha una funzione che dipende da essa come function4? Uso un driver per simulare la function1 e avrò un sostituto della function4 che mi garantisce che la function4 sia giusta e che quindi eventuali errori non derivino da lei.

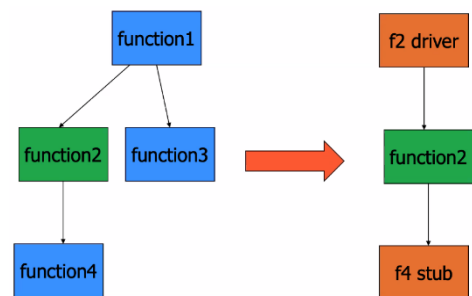
Driver: funzione/classe sviluppata all'interno del test per chiamare la funzione da testare.

Stub: creo una fake unit che mi aiuta a isolare la function2 dalle problematiche di function4 che abbia un risultato noto.

Driver e stub sono chiamati **mock ups**, o mock.

Quindi istanziamo da zero ogni volta l'oggetto Driver (guarda slides Converter).

Esporto Converter, faccio la new all'inizio di ogni test.



2 unità vengono testate isolate e poi una alla volta si aggiungono → **Integrazione incrementale**

Pro: mi accorgo facilmente di quale unità dà i problemi

Contro: spreco di risorse

➔ Noi dovremmo applicarlo. Magari non tutti ma avere un buon livello di integrazione è meglio. Questo con una buona coverage è il modo giusto.

REGRESSION TEST

- Serve a garantire che modifiche non introducano nuovi difetti.
- Riuso automatico dei test precedenti.

SYSTEM TESTING

- Test di tutti i moduli integrati, sia funzionali che non funzionali.
- Esegue test anche su piattaforme target reali (beta testing, acceptance testing).
- Considera usage profiles (funzioni più usate) e non functional requirements (efficienza, sicurezza...).

Con questo test mi focalizzo su tutte le proprietà, funzionali e non funzionali

	Functional/non functional	Who tests	Platform	Techniques
Unit test	Functional / structural	Developer or test group	development	BB, WB
Integration test	Functional	Developer or test group	development	Incremental TD or BU
System test	Functional + non functional	Developer or test group or user	development, target	Requirement coverage Use cases coverage Usage profiles Nf properties

ESAME: Teoria

Risk base → sistem test che cerca di testare il rischio

Reliability & Risk-Based Testing

- Reliability: misura la probabilità di failure nel tempo (MTBF, defect rate).
- Risk-based: identifica e testa le parti più critiche (es. ABS auto, sistemi medici).

Automazione dei test → Test documentati e automatizzati → riduzione errori e tempi.

MUTATION TESTING

Il **mutation testing** (o mutation analysis) è una tecnica utilizzata per valutare la qualità di un test suite (cioè un insieme di test) misurando quanto bene riesce a trovare (o "uccidere") versioni difettose del programma chiamate *mutanti*.

- Mutant (Mutante): È una versione del programma originale a cui è stata fatta una piccola modifica (una "mutazione") automatica, come ad esempio: cambiare un + in -, sostituire > con >=, cambiare una costante. Queste mutazioni simulano errori che un programmatore potrebbe fare.

- Killable Mutant (Mutante uccidibile): È un mutante che si comporta diversamente dal programma originale in almeno un caso. → Un buon test dovrebbe accorgersene e fallire, quindi "uccidere" il mutante.
 - Equivalent Mutant (Mutante equivalente): È un mutante che, nonostante la modifica, si comporta esattamente come il programma originale in tutti i casi. → Nessun test potrà mai ucciderlo, perché è funzionalmente equivalente all'originale.
 - Mutation Score: È un numero che misura l'efficacia del test suite.
- L'obiettivo è ottenere 100%, cioè che tutti i mutanti non equivalenti vengano uccisi dai test.

GUI TESTING

- Software GUI: qualsiasi software dotato di GUI (interfaccia utente grafica).
- GUI testing: è la pratica di testare un'applicazione software attraverso la sua interfaccia utente.

Test funzionali: test black box che esercitano le funzionalità di base di un'applicazione tramite l'interazione con la GUI, senza conoscere il codice sorgente

A cosa mi serve?

- Verifica dell'aspetto grafico: l'interfaccia utente grafica (GUI) deve apparire come definito nel documento dei requisiti o nei mockup forniti dagli stakeholder.
- Test di compatibilità: verificare che l'applicazione sia distribuita e funzioni correttamente su diversi dispositivi, schermi e layout (diversità di dispositivi).

Come li faccio?

- **Manual**: provo a vedere se la GUI funziona a ogni cosa che faccio
- **Scripted**: lancio dei pezzi di codici che scrivo io per verificare l'uso della GUI
- **Capture & Replay**: CAPTURE qualcuno registra e lo fa ripartire REPLAY finché il sistema non si comporta in modo diverso. Oggi non si usa più perché da errore x piccole variazioni.
- **Model-based**: fanno un modello della GUI (costoso e tanto tempo) ma non sbaglia più
- **Visual / Image Recognition**: se premo il bottone, parte quello che richiede il bottone. C'è un flusso, se si preme il bottone ESC allora si esce. Resistente alle variazioni di schermo.

TABLE BASED

Sono modi diversi di scrivere il test, abbiamo tabelle wiki linkate all'applicazione e poi il codice lo scrive da solo. Scrivo test case in maniera semi-automatica.

Esistono tools che mi dicono le performance, ma noi non riusciamo a vederle.

PROCESSES

PRINCIPALI MODELLI DI PROCESSO

WATERFALL

- Modello sequenziale con un'unica iterazione “lunga”: ogni fase produce un documento che viene “congelato” prima di passare alla successiva.
- Punti di forza: adatto a grandi progetti distribuiti, compliance a standard (ISO 12207, CMM-I).
- Limiti: scarsa flessibilità, costi elevati per cambiamenti tardivi (specie requisiti e design) .

Varianti del Waterfall

- V-Model: enfatizza attività di Verifica e Validazione (unit/integration/acceptance test) corrispondenti alle fasi di requirements e design.
- Prototipazione + Waterfall: costruzione rapida di prototipo per chiarire i requisiti, poi processo waterfall tradizionale .

MODELLO INCREMENTALE

Suddivide l'integrazione in più “build” rilasciate progressivamente, offrendo feedback precoce e dilazionando dipendenze esterne .

Riuso di componenti

- Attività di ricerca, selezione e integrazione di componenti (open/closed source).
- Impatto su requisiti e design: serve adattare specifiche al “cosa è già disponibile” .

METODOLOGIE AGILI

MANIFESTO AGILE

valori: persone e interazioni > processi e strumenti; software funzionante > documentazione; collaborazione col cliente > negoziazione di contratto; risposta al cambiamento > seguire un piano .

PRINCIPI AGILE: Consegna continua di software di valore, accoglienza del cambiamento in ogni fase, rilascio frequente (settimane o mesi), collaborazione continua tra business e sviluppo, team auto-organizzati .

SCRUM

- Ruoli: Scrum Master, Product Owner, Development Team.
- Artefatti: Product Backlog, Sprint Backlog, Increment.
- Cerimonie: Daily Stand-up (15'), Sprint Planning (1 giorno), Sprint Review (demo, max 4 h), Retrospective.
- Kanban su GitLab Issues: etichette Backlog, ToDo, Doing, Done .

EXTREME PROGRAMMING (XP)

- Valori: comunicazione, semplicità, feedback rapido, coraggio.
- Pratiche “estreme”: test-first (TDD), pair programming, refactoring continuo, design semplice “per l’oggi”, integrazione continua, proprietà collettiva del codice .

Le 12 pratiche XP

1. On-site Customer
2. Small Releases
3. Metaphor (architettura condivisa)
4. Testing (unit e acceptance)
5. Simple Design
6. Refactoring
7. Pair Programming
8. Planning Game
9. Sustainable Pace
10. Collective Code Ownership
11. Continuous Integration
12. Coding Standards

Criticità nell’adozione: Richiesta di co-locazione del team, modello contrattuale time-and-material, scalabilità limitata a piccoli gruppi (2–10 persone), mancanza di specialisti dedicati .

PROGETTO GeoControl – TypeScript e ORM

Cosa è TypeScript?

- Superset di JavaScript con tipizzazione statica opzionale.
- Va transpilato in JavaScript (non è interpretato né compilato nativamente).
- Supporta paradigmi multipli: OOP, funzionale, generico.

Perché usarlo?

- Controlli statici = meno bug a runtime.
- Inferenza di tipo = meno boilerplate.
- Refactoring sicuro grazie a IDE tipo VSCode.
- Perfetto per grandi codebase, come nei progetti di ingegneria del software.

TIPI PRIMITIVI

Non ha metodi associati e non è un oggetto. Tutte le primitive sono immutabili cioè il risultato che viene fuori da un'operazione wrapper non alterano il valore. Sono in minuscolo.

- string – textual data (e.g. "hello")
- number – integers and floats (e.g. 42, 3.14)
- boolean – true or false values (e.g. true)
- null – intentional absence of any value
- undefined – uninitialized value
- symbol – unique, immutable identifiers
- bigint – large integers beyond Number.MAX_SAFE_INTEGER

Duck Typing: Basato sulla struttura, non sul nome del tipo.

Java se abbiamo due classi identiche tranne per il nome è possibile farlo, si basa sulla struttura e non sulla parte nominale...idem TypeScript.

LITERAL TYPE

è un sottoinsieme di tipo primitivo per cui accettiamo solo alcuni valori specifici. Sono artifici del compilatore ma a run time non esistono più.

```
type O = 'a' | 'b' | 'c';
```

ENUM

In TypeScript, un enum è un insieme di valori costanti denominati. Sostanzialment sono come i literal ma a run time continuano a esistere dopo la traduzione in Java.

```
enum Color {  
    Red = '#ff0000',  
    Green = '#00ff00',  
    Blue = '#0000ff', }  

```

GENERICICS

I generici consentono di creare componenti e funzioni riutilizzabili che possono funzionare con più tipi perché non andiamo a specificare un tipo. Consentono di rendere il codice più flessibile e riutilizzabile.

ANY

Un valore può essere di un tipo o di un altro, non dobbiamo sceglierlo prima. Non ho vincoli.
`let something: any = "Hello world";`

UNION

Possiamo definire dei tipi che sono il risultato dell'unione di più tipi. Il tipo risultato avrà uno dei due tipi ma non insieme.

`type MyType = string | number;`

INTERSECTION

Definiamo due tipi con due proprietà, la loro Intersection è l'unione dei due oggetti. Il tipo risultato avrà entrambe le proprietà. (Union avrà o uno o l'altro).

`type CombinedType = TypeA & TypeB;`

DECORATORS

Quelle che in Java sono le istruzioni del compilatore, sono funzioni. Li useremo nella definizione delle entities del database.

OBJECT ORIENTED

Supporta la programmazione ad oggetti, useremo classi e interfacce ecc.

Diverse **visibilità**: public, private e protected. Esattamente come Java.

Costruttore: di default oppure no, crea un'istanza dell'oggetto.

NOTA: possiamo definire manualmente l'attributo o usare il modificatore di visibilità.

Variabili e metodi possono essere **statici** e si accede direttamente con la classe.

Interfacce possono essere usati come tipo e creare un oggetto usando gli attributi dell'interfaccia (in Java devo per forza creare prima la classe).

Parametri opzionali: Il punto di domanda dice che il parametro è opzionale. Nella signature del metodo vanno alla fine altrimenti avremmo un problema, se ce ne sono di più l'ordine è importante. Il linguaggio interpreta i valori in maniera sequenziale quindi se uno manca va messo null. ATTENZIONE: Nelle classi ha un valore diverso, vuol dire che può essere undefined MA deve per forza esistere.

Readonly, possiamo avere proprietà in sola lettura.

Dizionari sono coppie di chiave-valore, per ottimizzare una struttura.

Overloading si possono usare ma vanno definiti anche nella parte di implementazione con il tipo union. Cioè ho due funzioni uguali con diverso return, l'implementazione avrà una union tra i due tipi di ritorno. Non viene tradotto, serve al compilatore.

Polimorfismo consente di scrivere codice che funziona con tipi diversi tramite un'interfaccia condivisa. Il **Polimorfismo Strutturale** sfrutta il concetto di Union Type per creare una funzione polimorfa che va a chiamare una proprietà comune tra due oggetti senza definire ereditarietà.

SINCRONIA DI TYPESCRIPT

Concetto base: **Promise** → Promessa di eseguire un'operazione.

Java è monothread, con le promise possiamo procedere anche se altro sta in esecuzione.

Si definisce un codice da eseguire a secondo del risultato della promise.

Promise può essere tipicizzata e può avere un risultato di successo o reject.

NOTA: Non useremo Promise direttamente ma useremo async/await

Async/await

Async → attributo di una funzione (avrà una istr asincrona), restituirà una Promise

Await → modificatore da mettere davanti l'istruzione che contiene il parametro async

Una promise può essere anche void.

```
async function getUser(): Promise<void> {  
  const response: Response = await fetch("/api/user");  
  const data: { name: string } = await response.json();  
  console.log(data.name);  
}
```

Promise.All ci permette di parallelizzare due chiamate asincrone, sostanzialmente diciamo “Falle insieme e aspetta però che entrambe finiscano”. Possiamo fare quindi una await al posto di un ciclo for e chiamare in blocco le funzioni che sono nel loop.

Total time = tempo dell'operazione/istruzione più lenta.

ERRORI

Analoga a Java, con blocco try/catch. Lo useremo con le Promise in modo un poco diverso ma con await funziona come un normale blocco.

Throwing errors → sempre meglio avere una classe che estende la classe Error

ORM – Object Relational Mapping

Vogliamo che i dati persistano MA la memoria è volatile → Database

È una tecnica (e uno strumento) che permette di interagire con un databaseSQL.

C'è quindi un layer di astrazioni che ci fa lavorare con gli oggetti e non con i record.

ENTITY

Mappatura logica della entry del DB e il nostro oggetto.

Dobbiamo definire bene la classe pensando che questa corrisponderà ad un record.

Gli attributi della classe sono quindi le colonne di un record nel mio DB. Customizzazione.

Ereditarietà anche qui e ci sono diversi metodi per relazionarli.

Quando leggiamo un'entità possiamo definire una relazione tra le tabelle:

- Uno a uno (1:1) – Ogni istanza di entità è mappata esattamente a un'entità correlata
- Uno a molti (1:N) – Un'entità è correlata a più entità
- Molti a uno (N:1) – Più entità fanno riferimento alla stessa entità correlata
- Molti a molti (N:N) – Richiede una tabella di join per stabilire la relazione

Gli ORM consentono di configurare la cascata per tipo di operazione:

- Persist: salva le entità correlate quando il genitore viene salvato.
- Merge: aggiorna le entità correlate quando il genitore viene aggiornato.
- Remove: elimina le entità correlate quando il genitore viene eliminato.
- Detach: scollega le entità correlate quando il genitore viene rimosso dal contesto.
- Refresh: ricarica le entità correlate quando il genitore viene aggiornato.

Strategie di cascata:

- Cascata completa: applica tutte le operazioni alle entità correlate.
- Cascata selettiva: consente di propagare solo operazioni specifiche.
- Nessuna cascata: le relazioni devono essere gestite manualmente.

Loading strategies:

- Lazy Loading: i dati correlati vengono caricati solo quando si accede esplicitamente
- Eager Loading: i dati correlati vengono caricati immediatamente con l'entità

TYPE ORM

È il framework per TypeScript e JavaScript che useremo per non avere SQL nativo.

DataSource = Database “astratto” con cui interagiamo per recuperare i dati.

Persiste per tutta la vita del software, la connessione va distrutta quando chiudiamo il software. Avrà dei file dedicati per la creazione e distruzione.

Connessione attraverso dei parametri:

- Database type :SQLite
- Database name e eventuale credenziale di accesso
- La cartella delle entities (location)
- Policy di creazione e sincronizzazione
- Logging

SQLite: DBMS ma non supporta le credenziali di accesso, possono accedere tutti.

2 modalità di funzionamento:

- File based: persiste i dati su un file apposito
- In memory: salva i dati su una memoria volatile (lo useremo per i test)

ENTITIES

@Entity() definisce le tabelle

@Column() definisce i campi della tabella

- nullable: se un campo può essere NULL, valore predefinito = false
- unique: impone l'unicità su una colonna, valore predefinito = false
- default: imposta un valore predefinito per la colonna

@PrimaryColumn() definisce il campo chiave primaria di un'entità (id della entity)

- generated: strategia di generazione delle chiavi “increment” | “uuid” | “rowid” | “identity”
- type: specifica il tipo di colonna per le chiavi primarie personalizzate (“uuid”, “varchar”)

NOTA: Non tutto deve essere persistito.

RELATIONSHIPS

Si usa un “Decorator” che vuole dei parametri:

- TargetEntity: la classe dell'entità correlata
- Target.property: la proprietà nell'entità correlata che fa riferimento a questa entità (necessaria solo nelle relazioni bidirezionali)
- Options: definisce il comportamento della relazione (a cascata, regole di eliminazione, strategia di caricamento, ecc.)

Options:

- **Cascade Options** (cascade: true | [“insert”, “update”, “remove”])
 - true: applies to insert, update, and delete
 - “insert”: saves related entities automatically when the parent is saved
 - “update”: propagates updates to related entities when the parent is updated

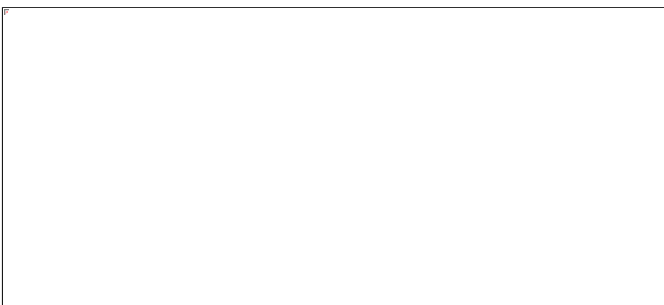
- "remove": deletes related entities when the parent is deleted
- Handled by TypeORM in memory before committing changes to the database
- **Delete Behavior** (onDelete)
 - "CASCADE": deletes related records automatically when the parent is deleted
 - "SET NULL": sets the foreign key to NULL instead of deleting related records
 - "RESTRICT": prevents deletion if related records exist
 - Handled by the database itself, enforcing constraints at the SQL level
- Strategia di caricamento **eager**: true
 - L'entità correlata viene caricata automaticamente quando viene caricata l'entità padre
 - La proprietà deve avere il tipo Entity
 - Non può essere utilizzata su entrambi i lati di una relazione bidirezionale
- Strategia di caricamento **lazy**: true
 - Il caricamento lazy è l'impostazione predefinita se eager non è specificato
 - La proprietà deve avere il tipo Promise<Entity>
 - L'entità correlata viene recuperata solo quando vi si accede

OneToOne

- Decorator: @OneToOne()
- Requires @JoinColumn() on the owning side to define the foreign key
- Supported options:
 - cascade: true | ["insert", "update", "remove"]
 - onDelete: "CASCADE" | "SET NULL" | "RESTRICT"

OneToMany

- Decorator: @OneToMany()
- Inverse side of @ManyToOne.
- No @JoinColumn() is needed, as the foreign key is in the other table
- @OneToMany is just a virtual mapping for easier querying
- For the latter reason, it doesn't support specific relationship option



ManyToMany

- Decorator: @ManyToMany()
- Requires @JoinTable() on the owning side
- TypeORM automatically creates a junction table to manage relationships
- Supported options:
 - cascade: true | ["insert", "update", "remove"]
 - No onDelete support (handled via constraints on the junction table)

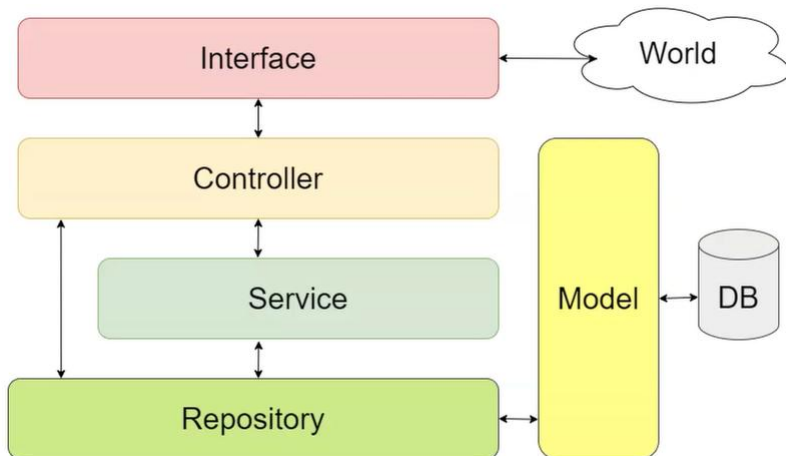
BIDIREZIONALE: Stessa politica da entrambi i versi. **ATTENZIONE:** un lato deve essere LAZY.

REPOSITORIES

Andremo a definire dei repository per ogni entity con l'idea di fare ricerca e utilizzare i metodi (create,update,delete o altro) sulla classe associata.

Quindi lo faremo per avere una buona separazione tra il livello del codice e quello del database passando per questo livello di mezzo.

ORM-based Layered Structure



RECAP SYSTEM DIGRAM E DEPLOYMENT

Partiamo dal context diagram.

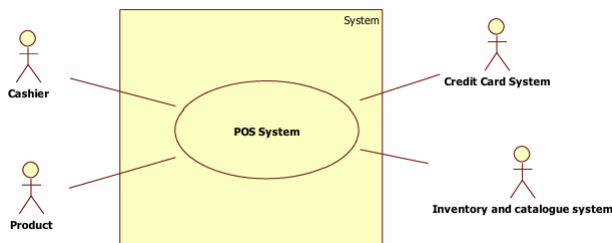
System diagram: tutto quello che è all'interno della BB del context diagram

Deployment diagram: tutto quello che ho nel context diagram, parti hardware e software del mio sistema

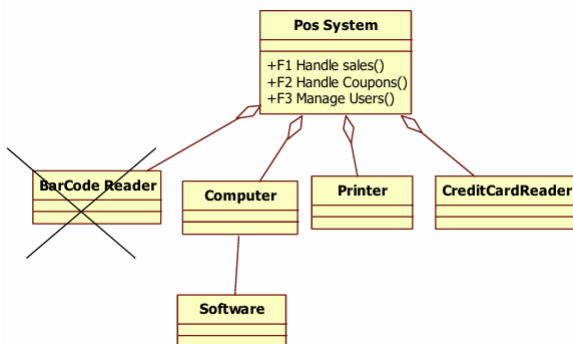
NOTA: Quando faccio il deployment diagram e ho già il system diagram, le parti software del system diagram diventano gli artefatti del deployment mentre le parti hardware diventano i nodi.

Vediamo un esempio:

CONTEXT DIAGRAM



SYSTEM DIAGRAM: solo quello che è all'interno della BB



Mettere BarCodeReader è un errore, è all'esterno della BB quindi non va messo.

DEPLOYMENT DIAGRAM: guardo il system diagram e vedo le parti sw/hw

Le parti hardware sono i nodi, quelle software sono gli artefatti. Tutto ciò che non è hw/sw non lo vado a mettere.

